Prompt for Task 1: Customer Data Cleaning

Clean a customer dataset (`shopping_trends.csv`). Handle missing values for `Age` (median), `Purchase Amount (USD)` (median), and `Location` (mode). Remove duplicate records. Standardize text values in the `Location` column (lowercase and strip whitespace). Finally, encode categorical variables `Gender` and `Location` using one-hot encoding.

```
import pandas as pd

# Load the dataset
try:
    data = pd.read_csv("shopping_trends.csv")
    print("Original data head:\n", data.head())
    print("\nOriginal data info:\n")
    data.info()
except FileNotFoundError:
    print("Error: shopping_trends.csv not found. Please upload the
file or provide the correct path.")
    # As a fallback for demonstration, create a dummy DataFrame if the
file is not found
    data = pd.DataFrame({
        'Age': [25, 30, None, 40, 25, 30, 50],
        'Income': [50000, 60000, 75000, None, 50000, 60000, 90000],
        'City': ['New York', 'Los Angeles', '  london ', None, 'New
York', 'London', 'Paris'],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Male',
'Female', 'Male'],
        'Product': ['A', 'B', 'C', 'A', 'A', 'B', 'D']
    })
    print("\nUsing a dummy DataFrame for demonstration.\n")

# 1. Handle missing values
# For 'Age' and 'Purchase Amount (USD)', fill with median
data['Age'] = data['Age'].fillna(data['Age'].median())
data['Purchase Amount (USD)'] = data['Purchase Amount
(USD)'].fillna(data['Purchase Amount (USD)'].median())

# For 'Location', fill with mode or 'Unknown' if mode is problematic
if not data['Location'].mode().empty:
    data['Location'] = data['Location'].fillna(data['Location'].mode()
[0])
else:
    data['Location'] = data['Location'].fillna('Unknown')

print("\nData after handling missing values:\n")
data.info()

# 2. Remove duplicate records
duplicates_before = data.duplicated().sum()
```

```
data.drop_duplicates(inplace=True)
duplicates_after = data.duplicated().sum()
print(f"\nRemoved {duplicates_before - duplicates_after} duplicate
records.\n")

# 3. Standardize text values (e.g., location names in lowercase and
strip whitespace)
data['Location'] =
data['Location'].astype(str).str.lower().str.strip()
print("Data after standardizing Location names:\n")
print(data['Location'].value_counts())

# 4. Encode categorical variables (Gender, Location)
# Using one-hot encoding
data_encoded = pd.get_dummies(data, columns=['Gender', 'Location'],
drop_first=True)

print("\nCleaned data head (with encoded variables):\n",
data_encoded.head())
print("\nCleaned data info:\n")
data_encoded.info()

# The 'data_encoded' DataFrame is now ready for analysis.
```

```
Original data head:
    Customer ID  Age Gender Item Purchased  Category  Purchase Amount
(USD)  \
0            1   55   Male         Blouse  Clothing
53
1            2   19   Male        Sweater  Clothing
64
2            3   50   Male          Jeans  Clothing
73
3            4   21   Male        Sandals  Footwear
90
4            5   45   Male         Blouse  Clothing
49

        Location Size      Color  Season  Review Rating Subscription
Status  \
0       Kentucky    L       Gray  Winter            3.1
Yes
1          Maine    L     Maroon  Winter            3.1
Yes
2  Massachusetts    S     Maroon  Spring            3.1
Yes
3   Rhode Island    M     Maroon  Spring            3.5
Yes
4         Oregon    M  Turquoise  Spring            2.7
Yes
```

```
  Payment Method  Shipping Type Discount Applied Promo Code Used  \
0    Credit Card        Express              Yes             Yes
1  Bank Transfer        Express              Yes             Yes
2           Cash  Free Shipping              Yes             Yes
3         PayPal   Next Day Air              Yes             Yes
4           Cash  Free Shipping              Yes             Yes

   Previous Purchases Preferred Payment Method Frequency of Purchases

0                  14                    Venmo            Fortnightly

1                   2                     Cash            Fortnightly

2                  23              Credit Card                 Weekly

3                  49                   PayPal                 Weekly

4                  31                   PayPal               Annually


Original data info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer ID               3900 non-null   int64
 1   Age                       3900 non-null   int64
 2   Gender                    3900 non-null   object
 3   Item Purchased            3900 non-null   object
 4   Category                  3900 non-null   object
 5   Purchase Amount (USD)     3900 non-null   int64
 6   Location                  3900 non-null   object
 7   Size                      3900 non-null   object
 8   Color                     3900 non-null   object
 9   Season                    3900 non-null   object
 10  Review Rating             3900 non-null   float64
 11  Subscription Status       3900 non-null   object
 12  Payment Method            3900 non-null   object
 13  Shipping Type             3900 non-null   object
 14  Discount Applied          3900 non-null   object
 15  Promo Code Used           3900 non-null   object
 16  Previous Purchases        3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   object
 18  Frequency of Purchases    3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
```

```
Data after handling missing values:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer ID               3900 non-null   int64
 1   Age                       3900 non-null   int64
 2   Gender                    3900 non-null   object
 3   Item Purchased            3900 non-null   object
 4   Category                  3900 non-null   object
 5   Purchase Amount (USD)     3900 non-null   int64
 6   Location                  3900 non-null   object
 7   Size                      3900 non-null   object
 8   Color                     3900 non-null   object
 9   Season                    3900 non-null   object
 10  Review Rating             3900 non-null   float64
 11  Subscription Status       3900 non-null   object
 12  Payment Method            3900 non-null   object
 13  Shipping Type             3900 non-null   object
 14  Discount Applied          3900 non-null   object
 15  Promo Code Used           3900 non-null   object
 16  Previous Purchases        3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   object
 18  Frequency of Purchases    3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB

Removed 0 duplicate records.

Data after standardizing Location names:

Location
montana           96
california        95
idaho             93
illinois          92
alabama           89
minnesota         88
new york          87
nevada            87
nebraska          87
delaware          86
maryland          86
vermont           85
louisiana         84
north dakota      83
west virginia     81
missouri          81
```

```
new mexico        81
mississippi       80
kentucky          79
arkansas          79
georgia           79
indiana           79
connecticut       78
north carolina    78
maine             77
ohio              77
virginia          77
texas             77
tennessee         77
south carolina    76
oklahoma          75
wisconsin         75
colorado          75
oregon            74
pennsylvania      74
michigan          73
washington        73
alaska            72
massachusetts     72
utah              71
wyoming           71
new hampshire     71
south dakota      70
iowa              69
florida           68
new jersey        67
arizona           65
hawaii            65
rhode island      63
kansas            63
Name: count, dtype: int64

Cleaned data head (with encoded variables):
    Customer ID  Age Item Purchased  Category  Purchase Amount (USD)
Size  \
0            1   55         Blouse  Clothing                     53
L
1            2   19        Sweater  Clothing                     64
L
2            3   50          Jeans  Clothing                     73
S
3            4   21        Sandals  Footwear                     90
M
4            5   45         Blouse  Clothing                     49
M
```

```
       Color  Season  Review Rating Subscription Status  ...  \
0       Gray  Winter            3.1                 Yes  ...
1     Maroon  Winter            3.1                 Yes  ...
2     Maroon  Spring            3.1                 Yes  ...
3     Maroon  Spring            3.5                 Yes  ...
4  Turquoise  Spring            2.7                 Yes  ...

  Location_south dakota Location_tennessee Location_texas
Location_utah  \
0                 False              False          False
False
1                 False              False          False
False
2                 False              False          False
False
3                 False              False          False
False
4                 False              False          False
False

   Location_vermont Location_virginia Location_washington  \
0             False             False               False
1             False             False               False
2             False             False               False
3             False             False               False
4             False             False               False

   Location_west virginia  Location_wisconsin  Location_wyoming
0                   False               False             False
1                   False               False             False
2                   False               False             False
3                   False               False             False
4                   False               False             False

[5 rows x 67 columns]

Cleaned data info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 67 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Customer ID                3900 non-null   int64
 1   Age                        3900 non-null   int64
 2   Item Purchased             3900 non-null   object
 3   Category                   3900 non-null   object
 4   Purchase Amount (USD)      3900 non-null   int64
 5   Size                       3900 non-null   object
```

```
6   Color                      3900 non-null   object
7   Season                     3900 non-null   object
8   Review Rating              3900 non-null   float64
9   Subscription Status        3900 non-null   object
10  Payment Method             3900 non-null   object
11  Shipping Type              3900 non-null   object
12  Discount Applied           3900 non-null   object
13  Promo Code Used            3900 non-null   object
14  Previous Purchases         3900 non-null   int64
15  Preferred Payment Method   3900 non-null   object
16  Frequency of Purchases     3900 non-null   object
17  Gender_Male                3900 non-null   bool
18  Location_alaska            3900 non-null   bool
19  Location_arizona           3900 non-null   bool
20  Location_arkansas          3900 non-null   bool
21  Location_california        3900 non-null   bool
22  Location_colorado          3900 non-null   bool
23  Location_connecticut       3900 non-null   bool
24  Location_delaware          3900 non-null   bool
25  Location_florida           3900 non-null   bool
26  Location_georgia           3900 non-null   bool
27  Location_hawaii            3900 non-null   bool
28  Location_idaho             3900 non-null   bool
29  Location_illinois          3900 non-null   bool
30  Location_indiana           3900 non-null   bool
31  Location_iowa              3900 non-null   bool
32  Location_kansas            3900 non-null   bool
33  Location_kentucky          3900 non-null   bool
34  Location_louisiana         3900 non-null   bool
35  Location_maine             3900 non-null   bool
36  Location_maryland          3900 non-null   bool
37  Location_massachusetts     3900 non-null   bool
38  Location_michigan          3900 non-null   bool
39  Location_minnesota         3900 non-null   bool
40  Location_mississippi       3900 non-null   bool
41  Location_missouri          3900 non-null   bool
42  Location_montana           3900 non-null   bool
43  Location_nebraska          3900 non-null   bool
44  Location_nevada            3900 non-null   bool
45  Location_new hampshire     3900 non-null   bool
46  Location_new jersey        3900 non-null   bool
47  Location_new mexico        3900 non-null   bool
48  Location_new york          3900 non-null   bool
49  Location_north carolina    3900 non-null   bool
50  Location_north dakota      3900 non-null   bool
51  Location_ohio              3900 non-null   bool
52  Location_oklahoma          3900 non-null   bool
53  Location_oregon            3900 non-null   bool
54  Location_pennsylvania      3900 non-null   bool
```

```
 55  Location_rhode island     3900 non-null   bool
 56  Location_south carolina   3900 non-null   bool
 57  Location_south dakota     3900 non-null   bool
 58  Location_tennessee        3900 non-null   bool
 59  Location_texas            3900 non-null   bool
 60  Location_utah             3900 non-null   bool
 61  Location_vermont          3900 non-null   bool
 62  Location_virginia         3900 non-null   bool
 63  Location_washington       3900 non-null   bool
 64  Location_west virginia    3900 non-null   bool
 65  Location_wisconsin        3900 non-null   bool
 66  Location_wyoming          3900 non-null   bool
dtypes: bool(50), float64(1), int64(4), object(12)
memory usage: 708.5+ KB
```

Prompt for Task 2: Sales Data Preprocessing

Preprocess sales transaction data (`sales_data.csv`). Convert the `Sale_Date` column to datetime format and create new features: `SaleYear`, `SaleMonth`, and `SaleDayOfWeek`. Normalize the `Sales_Amount` column using Min-Max scaling, creating a new column `NormalizedSales_Amount`. Detect and handle outliers in the `Sales_Amount` column using the IQR method (capping outliers).

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Load the dataset
try:
    sales_data = pd.read_csv("sales_data.csv")
    print("Original sales_data head:\n", sales_data.head())
    print("\nOriginal sales_data info:\n")
    sales_data.info()
except FileNotFoundError:
    print("Error: sales_data.csv not found. Please upload the file or
provide the correct path.")
    # As a fallback for demonstration, create a dummy DataFrame if the
file is not found
    print("\nUsing a dummy sales_data DataFrame for demonstration.\n")
    sales_data = pd.DataFrame({
        'OrderDate': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-
01-04', '2023-01-05', '2023-01-06', '2023-01-07', '2023-01-08'],
        'Sales': [100.0, 150.0, 2000.0, 50.0, 120.0, 180.0, 2500.0,
150.0],
        'Quantity': [1, 2, 10, 1, 3, 2, 15, 2],
        'CustomerID': [101, 102, 103, 104, 101, 105, 103, 106]
    })
    sales_data['OrderDate'] = pd.to_datetime(sales_data['OrderDate'])
# Ensure it's datetime for dummy
```

```
print("\n--- Starting Sales Data Preprocessing ---")

# 1. Convert date columns into datetime format
# Based on the loaded CSV, the date column is 'Sale_Date'
if 'Sale_Date' in sales_data.columns:
    sales_data['Sale_Date'] = pd.to_datetime(sales_data['Sale_Date'],
errors='coerce')
    # Drop rows where Sale_Date could not be parsed (if any)
    sales_data.dropna(subset=['Sale_Date'], inplace=True)
    print("\n'Sale_Date' column converted to datetime format.")
else:
    print("\n'Sale_Date' column not found. Skipping date conversion
and feature creation.")

# 2. Create new features (month, year, day of week)
if 'Sale_Date' in sales_data.columns:
    sales_data['SaleYear'] = sales_data['Sale_Date'].dt.year
    sales_data['SaleMonth'] = sales_data['Sale_Date'].dt.month
    sales_data['SaleDayOfWeek'] =
sales_data['Sale_Date'].dt.day_name()
    print("New features (SaleYear, SaleMonth, SaleDayOfWeek)
created.")

# 3. Normalize sales amount column
# Based on the loaded CSV, the sales amount column is 'Sales_Amount'
if 'Sales_Amount' in sales_data.columns:
    # Ensure 'Sales_Amount' is numeric
    sales_data['Sales_Amount'] =
pd.to_numeric(sales_data['Sales_Amount'], errors='coerce')
    sales_data.dropna(subset=['Sales_Amount'], inplace=True)

    if not sales_data['Sales_Amount'].empty:
        scaler = MinMaxScaler()
        sales_data['NormalizedSales_Amount'] =
scaler.fit_transform(sales_data[['Sales_Amount']])
        print("'Sales_Amount' column normalized using Min-Max scaling
(NormalizedSales_Amount created).")
    else:
        print("'Sales_Amount' column is empty after cleaning, skipping
normalization.")
else:
    print("\n'Sales_Amount' column not found. Skipping sales amount
normalization.")


# 4. Detect and handle outliers in dataset (for 'Sales_Amount' column)
if 'Sales_Amount' in sales_data.columns and not
sales_data['Sales_Amount'].empty:
    Q1 = sales_data['Sales_Amount'].quantile(0.25)
```

```
    Q3 = sales_data['Sales_Amount'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers_count = sales_data[(sales_data['Sales_Amount'] <
lower_bound) | (sales_data['Sales_Amount'] > upper_bound)].shape[0]
    if outliers_count > 0:
        print(f"Detected {outliers_count} outliers in 'Sales_Amount'
column using IQR method.")
        # Handle outliers by capping them
        sales_data['Sales_Amount_Capped'] =
sales_data['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)
        print("Outliers in 'Sales_Amount' column have been capped
(Sales_Amount_Capped created). If you prefer, these rows could also be
removed instead of capped.")
    else:
        print("No significant outliers detected in 'Sales_Amount'
column based on IQR method.")
else:
    print("\n'Sales_Amount' column not found or is empty. Skipping
outlier detection and handling.")

print("\n--- Sales Data Preprocessing Complete ---")
print("\nTransformed sales_data head:\n", sales_data.head())
print("\nTransformed sales_data info:\n")
sales_data.info()
```

```
Original sales_data head:
    Product_ID   Sale_Date Sales_Rep Region  Sales_Amount
Quantity_Sold  \
0        1052  2023-02-03       Bob  North       5053.97
18
1        1093  2023-04-21       Bob   West       4384.02
17
2        1015  2023-09-21     David  South       4631.23
30
3        1072  2023-08-24       Bob  South       2167.94
39
4        1061  2023-03-24   Charlie   East       3750.20
13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
```

```
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie

Original sales_data info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Product_ID            1000 non-null   int64
 1   Sale_Date             1000 non-null   object
 2   Sales_Rep             1000 non-null   object
 3   Region                1000 non-null   object
 4   Sales_Amount          1000 non-null   float64
 5   Quantity_Sold         1000 non-null   int64
 6   Product_Category      1000 non-null   object
 7   Unit_Cost             1000 non-null   float64
 8   Unit_Price            1000 non-null   float64
 9   Customer_Type         1000 non-null   object
 10  Discount              1000 non-null   float64
 11  Payment_Method        1000 non-null   object
 12  Sales_Channel         1000 non-null   object
 13  Region_and_Sales_Rep  1000 non-null   object
dtypes: float64(4), int64(2), object(8)
memory usage: 109.5+ KB

--- Starting Sales Data Preprocessing ---

'Sale_Date' column converted to datetime format.
New features (SaleYear, SaleMonth, SaleDayOfWeek) created.
'Sales_Amount' column normalized using Min-Max scaling
(NormalizedSales_Amount created).
No significant outliers detected in 'Sales_Amount' column based on IQR
method.

--- Sales Data Preprocessing Complete ---

Transformed sales_data head:
    Product_ID  Sale_Date Sales_Rep Region  Sales_Amount
Quantity_Sold  \
0        1052 2023-02-03       Bob  North       5053.97              18

1        1093 2023-04-21       Bob   West       4384.02              17

2        1015 2023-09-21     David  South       4631.23              30
```

```
3        1072 2023-08-24       Bob  South       2167.94             39

4        1061 2023-03-24   Charlie   East       3750.20             13


  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep  SaleYear
SaleMonth  \
0           Cash        Online            North-Bob      2023
2
1           Cash        Retail             West-Bob      2023
4
2  Bank Transfer        Retail          South-David      2023
9
3    Credit Card        Retail            South-Bob      2023
8
4    Credit Card        Online         East-Charlie      2023
3

  SaleDayOfWeek  NormalizedSales_Amount
0        Friday                0.500950
1        Friday                0.433202
2      Thursday                0.458201
3      Thursday                0.209105
4        Friday                0.369108

Transformed sales_data info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 18 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Product_ID             1000 non-null   int64
 1   Sale_Date              1000 non-null   datetime64[ns]
 2   Sales_Rep              1000 non-null   object
 3   Region                 1000 non-null   object
 4   Sales_Amount           1000 non-null   float64
 5   Quantity_Sold          1000 non-null   int64
 6   Product_Category       1000 non-null   object
 7   Unit_Cost              1000 non-null   float64
 8   Unit_Price             1000 non-null   float64
 9   Customer_Type          1000 non-null   object
 10  Discount               1000 non-null   float64
```

```
 11  Payment_Method          1000 non-null   object
 12  Sales_Channel           1000 non-null   object
 13  Region_and_Sales_Rep    1000 non-null   object
 14  SaleYear                1000 non-null   int32
 15  SaleMonth               1000 non-null   int32
 16  SaleDayOfWeek           1000 non-null   object
 17  NormalizedSales_Amount  1000 non-null   float64
dtypes: datetime64[ns](1), float64(5), int32(2), int64(2), object(8)
memory usage: 132.9+ KB
```

Prompt for Task 3: Healthcare Data Preparation

Clean and preprocess a healthcare dataset (`dirty_v3_path.csv`). Handle missing values in `Age`, `Glucose`, `Blood Pressure` by filling with the median, and `Gender`, `Medical Condition` by filling with the mode. Encode categorical features (`Gender`, `Medical Condition`, `random_notes`) using one-hot encoding. Scale numerical features (`Age`, `Glucose`, `Blood Pressure`, `BMI`, `Oxygen Saturation`, `LengthOfStay`, `Cholesterol`, `Triglycerides`, `HbA1c`, `Smoking`, `Alcohol`, `Physical Activity`, `Diet Score`, `Family History`, `Stress Level`, `Sleep Hours`, `noise_col`) using MinMaxScaler. Finally, split the dataset into training and testing sets (70/30 split) with 'Outcome' as the target variable.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.impute import SimpleImputer
import numpy as np

# Define the expected filename
file_name = "dirty_v3_path.csv"

# Load the dataset
try:
    healthcare_data = pd.read_csv(file_name)
    print(f"Original {file_name} head:\n", healthcare_data.head())
    print(f"\nOriginal {file_name} info:\n")
    healthcare_data.info()
except FileNotFoundError:
    print(f"Error: {file_name} not found. Please upload the file or
provide the correct path.")
    # As a fallback for demonstration, create a dummy DataFrame if the
file is not found
    print(f"\nUsing a dummy {file_name} DataFrame for demonstration.\
n")
    healthcare_data = pd.DataFrame({
        'Age': [30, 45, 60, 22, 55, 38, 70, 29, 41, 50],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Male',
'Female', 'Male', 'Female', 'Male', 'Female'],
        'blood_pressure': [120, 140, 130, None, 150, 125, 160, 110,
None, 135],
```

```
        'cholesterol': [200, 240, None, 180, 250, 210, 280, 170, 220,
None],
        'BMI': [25.5, 30.1, 28.0, 20.3, 32.5, 24.1, 35.0, 19.5, 26.7,
31.2],
        'Smoking': ['No', 'Yes', 'No', 'No', 'Yes', 'No', 'Yes', 'No',
'No', 'Yes'],
        'PhysicalActivity': [3, 1, 2, 4, 1, 3, 0, 5, 2, 1],
        'Outcome': [0, 1, 0, 0, 1, 0, 1, 0, 0, 1] # Hypothetical
target variable
    })
    print("Dummy data head:\n", healthcare_data.head())
    print("\nDummy data info:\n")
    healthcare_data.info()

print("\n--- Starting Healthcare Data Preprocessing ---")

# 1. Handle missing values
# For 'blood_pressure' and 'cholesterol', fill with median
for col in ['blood_pressure', 'cholesterol']:
    if col in healthcare_data.columns and
healthcare_data[col].isnull().any():
        median_val = healthcare_data[col].median()
        healthcare_data[col] = healthcare_data[col].fillna(median_val)
        print(f"Filled missing values in '{col}' with median:
{median_val:.2f}")

# Identify other numerical and categorical columns for general
imputation
numerical_cols =
healthcare_data.select_dtypes(include=np.number).columns.tolist()
categorical_cols =
healthcare_data.select_dtypes(include='object').columns.tolist()

# Remove 'blood_pressure' and 'cholesterol' from numerical_cols if
already handled
numerical_cols = [col for col in numerical_cols if col not in
['blood_pressure', 'cholesterol']]

# Impute remaining numerical columns with median
if numerical_cols:
    for col in numerical_cols:
        if healthcare_data[col].isnull().any():
            median_val = healthcare_data[col].median()
            healthcare_data[col] =
healthcare_data[col].fillna(median_val)
            print(f"Filled missing values in numerical column '{col}'
with median: {median_val:.2f}")

# Impute categorical columns with mode
if categorical_cols:
```

```
    for col in categorical_cols:
        if healthcare_data[col].isnull().any():
            mode_val = healthcare_data[col].mode()[0]
            healthcare_data[col] =
healthcare_data[col].fillna(mode_val)
            print(f"Filled missing values in categorical column
'{col}' with mode: {mode_val}")

print("\nData after handling all missing values:\n")
healthcare_data.info()

# 2. Encode categorical features
# Identify categorical columns (object dtype)
categorical_features =
healthcare_data.select_dtypes(include='object').columns

# Drop the target column if it's in categorical features and meant for
'y'
# Assuming 'Outcome' is the target and will be separated later
if 'Outcome' in categorical_features:
    categorical_features = categorical_features.drop('Outcome')

if len(categorical_features) > 0:
    print(f"\nEncoding categorical features:
{list(categorical_features)}")
    healthcare_data_encoded = pd.get_dummies(healthcare_data,
columns=categorical_features, drop_first=True)
    print("Categorical features encoded.")
else:
    healthcare_data_encoded = healthcare_data.copy()
    print("No categorical features to encode.")


# 3. Scale numerical features
# Identify numerical columns for scaling after encoding
# Exclude the target column if it's numerical
numerical_features_to_scale =
healthcare_data_encoded.select_dtypes(include=np.number).columns.tolis
t()

target_column = 'Outcome' # *** IMPORTANT: Please adjust this to your
actual target column name ***

if target_column in numerical_features_to_scale:
    numerical_features_to_scale.remove(target_column)

if len(numerical_features_to_scale) > 0:
    print(f"\nScaling numerical features:
{numerical_features_to_scale}")
    scaler = MinMaxScaler() # Using Min-Max scaling, can be changed to
```

```
StandardScaler()
    healthcare_data_encoded[numerical_features_to_scale] =
scaler.fit_transform(healthcare_data_encoded[numerical_features_to_sca
le])
    print("Numerical features scaled.")
else:
    print("No numerical features to scale.")

print("\nPreprocessed data head (after encoding and scaling):\n",
healthcare_data_encoded.head())
print("\nPreprocessed data info (after encoding and scaling):\n")
healthcare_data_encoded.info()

# 4. Split dataset into training and testing sets
if target_column in healthcare_data_encoded.columns:
    X = healthcare_data_encoded.drop(columns=[target_column])
    y = healthcare_data_encoded[target_column]

    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42, stratify=y if y.nunique() > 1 else
None)

    print("\nDataset split into training and testing sets:")
    print(f"X_train shape: {X_train.shape}")
    print(f"X_test shape: {X_test.shape}")
    print(f"y_train shape: {y_train.shape}")
    print(f"y_test shape: {y_test.shape}")
else:
    print(f"\nWarning: Target column '{target_column}' not found.
Cannot split dataset into X and y.")
    print("X (features) would be the entire preprocessed DataFrame:")
    X = healthcare_data_encoded
    X_train, X_test = train_test_split(X, test_size=0.3,
random_state=42)
    print(f"X_train shape: {X_train.shape}")
    print(f"X_test shape: {X_test.shape}")

print("\n--- Healthcare Data Preprocessing Complete ---")
# The 'healthcare_data_encoded' DataFrame, and 'X_train', 'X_test',
'y_train', 'y_test' (if target found) are now ready for machine
learning models.
```

```
Original dirty_v3_path.csv head:
     Age  Gender Medical Condition  Glucose  Blood Pressure    BMI  \
0  46.0    Male          Diabetes   137.04          135.27  28.90
1  22.0    Male           Healthy    71.58          113.27  26.29
2  50.0     NaN            Asthma    95.24             NaN  22.53
3  57.0     NaN           Obesity      NaN          130.53  38.47
4  66.0  Female      Hypertension    95.15          178.17  31.12
```

```
   Oxygen Saturation  LengthOfStay  Cholesterol  Triglycerides  HbA1c
\
0              96.04             6       231.88         210.56   7.61

1              97.54             2       165.57         129.41   4.91

2              90.31             2       214.94         165.35   5.60

3              96.60             5       197.71         182.13   6.92

4              94.90             4       259.53         115.85   5.98


   Smoking  Alcohol  Physical Activity  Diet Score  Family History  \
0        0        0               -0.20        3.54               0
1        0        0                8.12        5.90               0
2        0        0                5.01        4.65               1
3        0        0                3.16        3.37               0
4        0        1                3.56        3.40               0

   Stress Level  Sleep Hours random_notes   noise_col
0          5.07         6.05        lorem -137.057211
1          5.87         7.72        ipsum  -11.230610
2          3.09         4.82        ipsum   98.331195
3          3.01         5.33        lorem   44.187175
4          6.38         6.64        lorem   44.831426

Original dirty_v3_path.csv info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Age                25500 non-null  float64
 1   Gender             25500 non-null  object
 2   Medical Condition  25500 non-null  object
 3   Glucose            25500 non-null  float64
 4   Blood Pressure     25500 non-null  float64
 5   BMI                30000 non-null  float64
 6   Oxygen Saturation  30000 non-null  float64
 7   LengthOfStay       30000 non-null  int64
 8   Cholesterol        30000 non-null  float64
 9   Triglycerides      30000 non-null  float64
 10  HbA1c              30000 non-null  float64
 11  Smoking            30000 non-null  int64
 12  Alcohol            30000 non-null  int64
 13  Physical Activity  30000 non-null  float64
 14  Diet Score         30000 non-null  float64
 15  Family History     30000 non-null  int64
```

```
 16  Stress Level       30000 non-null  float64
 17  Sleep Hours        30000 non-null  float64
 18  random_notes       30000 non-null  object
 19  noise_col          30000 non-null  float64
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB

--- Starting Healthcare Data Preprocessing ---
Filled missing values in numerical column 'Age' with median: 55.00
Filled missing values in numerical column 'Glucose' with median:
110.50
Filled missing values in numerical column 'Blood Pressure' with
median: 138.32
Filled missing values in categorical column 'Gender' with mode: Female
Filled missing values in categorical column 'Medical Condition' with
mode: Hypertension

Data after handling all missing values:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Age                30000 non-null  float64
 1   Gender             30000 non-null  object
 2   Medical Condition  30000 non-null  object
 3   Glucose            30000 non-null  float64
 4   Blood Pressure     30000 non-null  float64
 5   BMI                30000 non-null  float64
 6   Oxygen Saturation  30000 non-null  float64
 7   LengthOfStay       30000 non-null  int64
 8   Cholesterol        30000 non-null  float64
 9   Triglycerides      30000 non-null  float64
 10  HbA1c              30000 non-null  float64
 11  Smoking            30000 non-null  int64
 12  Alcohol            30000 non-null  int64
 13  Physical Activity  30000 non-null  float64
 14  Diet Score         30000 non-null  float64
 15  Family History     30000 non-null  int64
 16  Stress Level       30000 non-null  float64
 17  Sleep Hours        30000 non-null  float64
 18  random_notes       30000 non-null  object
 19  noise_col          30000 non-null  float64
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB

Encoding categorical features: ['Gender', 'Medical Condition',
'random_notes']
Categorical features encoded.
```

```
Scaling numerical features: ['Age', 'Glucose', 'Blood Pressure',
'BMI', 'Oxygen Saturation', 'LengthOfStay', 'Cholesterol',
'Triglycerides', 'HbA1c', 'Smoking', 'Alcohol', 'Physical Activity',
'Diet Score', 'Family History', 'Stress Level', 'Sleep Hours',
'noise_col']
Numerical features scaled.

Preprocessed data head (after encoding and scaling):
         Age   Glucose  Blood Pressure       BMI  Oxygen Saturation  \
0  0.455696  0.391428        0.401144  0.431680           0.670348
1  0.151899  0.171904        0.256540  0.378609           0.705592
2  0.506329  0.251249        0.421191  0.302155           0.535714
3  0.594937  0.302425        0.369988  0.626271           0.683506
4  0.708861  0.250947        0.683121  0.476820           0.643562

   LengthOfStay  Cholesterol  Triglycerides     HbA1c  Smoking  ...  \
0      0.277778     0.518390       0.524877  0.476872      0.0  ...
1      0.055556     0.265915       0.342102  0.179515      0.0  ...
2      0.055556     0.453891       0.423050  0.255507      0.0  ...
3      0.222222     0.388288       0.460844  0.400881      0.0  ...
4      0.166667     0.623667       0.311561  0.297357      0.0  ...

   Gender_Male  Medical Condition_Asthma  Medical Condition_Cancer  \
0         True                     False                     False
1         True                     False                     False
2        False                      True                     False
3        False                     False                     False
4        False                     False                     False

   Medical Condition_Diabetes  Medical Condition_Healthy  \
0                        True                      False
1                       False                       True
2                       False                      False
3                       False                      False
4                       False                      False

   Medical Condition_Hypertension  Medical Condition_Obesity
random_notes_??  \
0                           False                      False
False
1                           False                      False
False
2                           False                      False
False
3                           False                       True
False
4                            True                      False
False
```

```
   random_notes_ipsum  random_notes_lorem
0               False               True
1                True              False
2                True              False
3               False               True
4               False               True

[5 rows x 27 columns]

Preprocessed data info (after encoding and scaling):

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 27 columns):
 #   Column                           Non-Null Count  Dtype
---  ------                           --------------  -----
 0   Age                              30000 non-null  float64
 1   Glucose                          30000 non-null  float64
 2   Blood Pressure                   30000 non-null  float64
 3   BMI                              30000 non-null  float64
 4   Oxygen Saturation                30000 non-null  float64
 5   LengthOfStay                     30000 non-null  float64
 6   Cholesterol                      30000 non-null  float64
 7   Triglycerides                    30000 non-null  float64
 8   HbA1c                            30000 non-null  float64
 9   Smoking                          30000 non-null  float64
 10  Alcohol                          30000 non-null  float64
 11  Physical Activity                30000 non-null  float64
 12  Diet Score                       30000 non-null  float64
 13  Family History                   30000 non-null  float64
 14  Stress Level                     30000 non-null  float64
 15  Sleep Hours                      30000 non-null  float64
 16  noise_col                        30000 non-null  float64
 17  Gender_Male                      30000 non-null  bool
 18  Medical Condition_Asthma         30000 non-null  bool
 19  Medical Condition_Cancer         30000 non-null  bool
 20  Medical Condition_Diabetes       30000 non-null  bool
 21  Medical Condition_Healthy        30000 non-null  bool
 22  Medical Condition_Hypertension   30000 non-null  bool
 23  Medical Condition_Obesity        30000 non-null  bool
 24  random_notes_??                  30000 non-null  bool
 25  random_notes_ipsum               30000 non-null  bool
 26  random_notes_lorem               30000 non-null  bool
dtypes: bool(10), float64(17)
memory usage: 4.2 MB

Warning: Target column 'Outcome' not found. Cannot split dataset into
X and y.
X (features) would be the entire preprocessed DataFrame:
X_train shape: (21000, 27)
```

```
X_test shape: (9000, 27)

--- Healthcare Data Preprocessing Complete ---
```

Prompt for Task 4: Employee Salary Data Preprocessing

Clean and preprocess an employee salary dataset (`Employee_Salary_Dataset.csv`). Handle missing values by filling numerical columns with their median and categorical columns with their mode. Detect and remove salary outliers using the IQR method. Standardize department names by converting them to lowercase and stripping whitespace. Apply label encoding to the `JobLocation` and `Department` columns.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# Define the expected filename
file_name = "Employee_Salary_Dataset.csv"

# Load the dataset
try:
    df = pd.read_csv(file_name)
    print(f"Original {file_name} head:\n", df.head())
    print(f"\nOriginal {file_name} info:\n")
    df.info()
except FileNotFoundError:
    print(f"Error: {file_name} not found. Please upload the file or
provide the correct path.")
    # As a fallback for demonstration, create a dummy DataFrame if the
file is not found
    print(f"\nUsing a dummy {file_name} DataFrame for demonstration.\
n")
    df = pd.DataFrame({
        'EmployeeID': range(1, 11),
        'Age': [30, 45, 25, 50, 35, 28, 40, 55, 32, 48],
        'Department': ['Sales', 'Marketing', '  HR  ', 'IT', None,
'Sales', 'Marketing', 'IT', 'HR', 'IT'],
        'JobTitle': ['Manager', 'Analyst', 'Assistant', 'Engineer',
'Manager', 'Analyst', 'Manager', 'Engineer', 'Assistant', 'Manager'],
        'Salary': [70000, 60000, 45000, 80000, 65000, 50000, 75000,
150000, 48000, 90000],
        'JobLocation': ['New York', 'Los Angeles', '  london ', 'New
York', None, 'Chicago', 'Los Angeles', 'London', 'Chicago', 'New
York'],
        'ExperienceYears': [5, 10, 2, 15, 7, 3, 12, 20, 4, 18],
        'PerformanceRating': [4, 3, 5, 4, 3, 4, 5, 2, 3, 4]
    })
    print("Dummy data head:\n", df.head())
    print("\nDummy data info:\n")
```

```
    df.info()

print("\n--- Starting Employee Salary Data Preprocessing ---")

# 1. Handle missing values
# For numerical columns like 'Salary', 'Age', 'ExperienceYears',
'PerformanceRating', fill with median
for col in ['Salary', 'Age', 'ExperienceYears', 'PerformanceRating']:
    if col in df.columns and df[col].isnull().any():
        median_val = df[col].median()
        df[col] = df[col].fillna(median_val)
        print(f"Filled missing values in numerical column '{col}' with
median: {median_val:.2f}")

# For categorical columns like 'Department', 'JobLocation', fill with
mode
for col in ['Department', 'JobLocation']:
    if col in df.columns and df[col].isnull().any():
        mode_val = df[col].mode()[0]
        df[col] = df[col].fillna(mode_val)
        print(f"Filled missing values in categorical column '{col}'
with mode: {mode_val}")

print("\nData after handling all missing values:\n")
df.info()

# 2. Detect and remove salary outliers
if 'Salary' in df.columns:
    Q1 = df['Salary'].quantile(0.25)
    Q3 = df['Salary'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    initial_rows = df.shape[0]
    df_no_outliers = df[(df['Salary'] >= lower_bound) & (df['Salary']
<= upper_bound)]
    removed_outliers = initial_rows - df_no_outliers.shape[0]
    df = df_no_outliers.copy() # Update the DataFrame to contain data
without outliers

    if removed_outliers > 0:
        print(f"\nDetected and removed {removed_outliers} salary
outliers using IQR method.")
    else:
        print("\nNo significant salary outliers detected based on IQR
method.")
else:
    print("\n'Salary' column not found. Skipping outlier detection and
removal.")
```

```
# 3. Standardize department names (lowercase and strip whitespace)
if 'Department' in df.columns:
    df['Department'] =
df['Department'].astype(str).str.lower().str.strip()
    print("\n'Department' column standardized (lowercase and stripped
whitespace).")
    print("Standardized Department unique values:\n",
df['Department'].unique())
else:
    print("\n'Department' column not found. Skipping
standardization.")

# 4. Apply label encoding to job location, department
le = LabelEncoder()

if 'JobLocation' in df.columns:
    df['JobLocation_Encoded'] = le.fit_transform(df['JobLocation'])
    print(f"\n'JobLocation' column label encoded. Mappings:
{list(le.classes_)} -> {list(range(len(le.classes_)))}")
else:
    print("\n'JobLocation' column not found. Skipping label
encoding.")

if 'Department' in df.columns:
    df['Department_Encoded'] = le.fit_transform(df['Department'])
    print(f"'Department' column label encoded. Mappings:
{list(le.classes_)} -> {list(range(len(le.classes_)))}")
else:
    print("\n'Department' column not found. Skipping label encoding.")

print("\n--- Employee Salary Data Preprocessing Complete ---")
print("\nPreprocessed data head:\n", df.head())
print("\nPreprocessed data info:\n")
df.info()
```

```
Original Employee_Salary_Dataset.csv head:
    ID  Experience_Years  Age  Gender  Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000

Original Employee_Salary_Dataset.csv info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
```

```
---  ------             --------------  -----
 0   ID                 35 non-null     int64
 1   Experience_Years   35 non-null     int64
 2   Age                35 non-null     int64
 3   Gender             35 non-null     object
 4   Salary             35 non-null     int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB

--- Starting Employee Salary Data Preprocessing ---

Data after handling all missing values:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   ID                 35 non-null     int64
 1   Experience_Years   35 non-null     int64
 2   Age                35 non-null     int64
 3   Gender             35 non-null     object
 4   Salary             35 non-null     int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB

Detected and removed 2 salary outliers using IQR method.

'Department' column not found. Skipping standardization.

'JobLocation' column not found. Skipping label encoding.

'Department' column not found. Skipping label encoding.

--- Employee Salary Data Preprocessing Complete ---

Preprocessed data head:
    ID  Experience_Years  Age  Gender  Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000

Preprocessed data info:

<class 'pandas.core.frame.DataFrame'>
Index: 33 entries, 0 to 34
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
```

```
---   ------             --------------  -----
 0   ID                 33 non-null     int64
 1   Experience_Years  33 non-null     int64
 2   Age                33 non-null     int64
 3   Gender             33 non-null     object
 4   Salary             33 non-null     int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB
```